Concurrency Fundamentals and Foundation Race-Condition PoC

Udit | EPAM CoE: Hardware Concurrency, Memory Models & Lock-Free Design | 2026-10-09

Evidence labels: TEAM-MEASURED, EXTERNAL EVIDENCE, THEORETICAL, STATICALLY REVIEWED, PENDING LOCAL EXECUTION.

1. Problem

Concurrency here means several threads making progress over overlapping time on shared mutable state: one variable that more than one thread can read and write. When nothing orders those accesses, the result depends on thread scheduling, not on the source code alone. This section establishes that problem with a small, reproducible Java experiment that later team sections build on. (THEORETICAL)

2. Race Condition: the Lost Update

In Java, `count++` is specified as: read the variable, add 1, store the result (JLS 15.14.2, EXTERNAL EVIDENCE). These are separate steps, so two threads can interleave them. The table is a conceptual interleaving (THEORETICAL), not a trace captured from a CPU.

Step	Thread A	Thread B	Shared counter
0			0
1	reads 0		0
2		reads 0	0
3	computes 1		0
4		computes 1	0
5	writes 1		1
6		writes 1	1

Final value is 1 but the expected value is 2: one update is lost. With expected = threads x increments per thread, a race appears as actual < expected.

3. Why Normal Testing Can Miss It

Thread scheduling is nondeterministic, so a race shows only on some interleavings. A passing test, or many passing tests, does not show that no race exists. In our data the broken counter passed 163 of 180 runs (Section 7). A conventional unit test checks one outcome per execution; a stress test repeats the workload under contention to raise the chance of hitting a bad interleaving; a litmus tool such as jcstress records which outcomes occur across very many executions (not run here, see Section 10). Our unit test for the unsafe counter therefore asserts only a sane range, never failure.

4. JMM / Hardware Boundary (High Level)

Java source, then the Java Memory Model (JLS Chapter 17), then JVM/JIT, then machine instructions, then CPU and caches. Correctness of a Java program is defined at the JLS level through three properties: atomicity (a read-modify-write happens as one indivisible step), visibility (when one thread's writes are seen by another) and ordering (happens-before). Code that reads sequentially guarantees none of these across threads. This experiment observes only Java-level results; it does not show which instructions the JIT emitted or what the CPU or caches did. Detail belongs to later members: JMM and barriers (Prachi), compiler/CPU reordering (Vanshraj), CAS (Aastha).

5. Implementation (STATICALLY REVIEWED; executed in Section 7)

Three classes implement one `Counter` interface; full code is in the repository under `src/main/java/fundamentals`.

```
UnsafeCounter:        count++;                                    // plain int
SynchronizedCounter:  public synchronized void increment() { count++; }
AtomicCounter:        count.incrementAndGet();                    //
AtomicInteger
```

`Workload` creates the threads, releases them together through a `CountDownLatch`, then the main thread joins them and reads the result (join orders the reads after the workers' writes, JLS 17.4.5).

6. Experiment Setup

- Workload: 8 threads; 1,000,000 or 10,000,000 increments per thread; expected 8,000,000 or 80,000,000; a fresh counter every run.
- Repetitions: 30 runs per implementation per JVM launch; 6 launches (4 at 1M, 2 at 10M). JUnit: 11 tests (8 threads x 200,000), executed twice.
- Archiving: output of 4 launches is in `results/raw`; the 2 launches of 2026-10-09 are console output only (not archived).
- Environment (TEAM-MEASURED): Intel Core i7-13700HX, 16 cores / 24 logical processors, 15.7 GB RAM, Windows 11 Home Single Language 10.0.26200, Oracle Java 27 (build 27+35-2325, HotSpot 64-Bit Server VM), Apache Maven 3.10.0.
- Reproduce: `mvn test`, then `java -cp target/classes fundamentals.RaceExperiment 8 1000000 30`.

7. Actual Results (TEAM-MEASURED, 2026-10-08 and 2026-10-09)

Implementation	Threads	Iterations	Expected	Runs	Incorrect	Correct
Unsafe	8	1,000,000	8,000,000	120	12	108
Unsafe	8	10,000,000	80,000,000	60	5	55
Synchronized	8	1,000,000	8,000,000	120	0	120
Synchronized	8	10,000,000	80,000,000	60	0	60
Atomic	8	1,000,000	8,000,000	120	0	120
Atomic	8	10,000,000	80,000,000	60	0	60

Totals: Unsafe 17 incorrect of 180 runs; Synchronized 0 of 180; Atomic 0 of 180.

Representative failures: Unsafe, 8 x 1,000,000, expected 8,000,000, actual 1,694,974 (FAIL). JUnit unsafe test, 8 x 200,000, expected 1,600,000: actual 268,174 and 278,009 in two executions. Across failing Unsafe runs, actual ranged from 1,694,974 to 6,549,002 (of 8,000,000) and from 19,948,701 to 60,000,000 (of 80,000,000).

Observation: all 17 Unsafe failures fell within the first three runs of their JVM launch; later runs in the same launch were correct. The cause is not established.

8. Findings

- The unsynchronized counter lost updates in 17 of 180 runs; in the worst runs more than three quarters of the increments were lost.
- The synchronized and atomic counters were correct in all 180 runs each.
- Failures were not evenly distributed over runs, so 17/180 is not a failure rate; runs within one JVM are not independent.
- The 11 JUnit tests passed with 0 failures even though one tests a broken class.

9. What the Experiment Proves

- Unsynchronized shared mutation can produce incorrect results, and the loss can be large (TEAM-MEASURED).
- In 180 runs each, synchronization and an atomic update produced the correct total. This is consistent with their documented guarantees (EXTERNAL EVIDENCE, Section 11); the measurement alone does not prove them correct.
- Passing tests alone do not establish the absence of a concurrency bug: 163 of 180 unsafe runs and all unit tests passed.

10. What It Does NOT Prove

- It does not prove any CPU-level instruction sequence or that the Section 2 interleaving occurred literally.
- It does not measure cache coherence, cache-line bouncing or false sharing.
- It does not establish a performance winner: no timing was collected.
- It does not reproduce detailed JMM behavior; jcstress was not run (PENDING LOCAL EXECUTION).
- It does not explain why failures clustered in early runs (open question for the JIT/reordering owners), and covers one machine and one JVM build.

11. References (EXTERNAL EVIDENCE; full claim table in docs/references.md)

1. Java Language Specification, 15.14.2 Postfix Increment Operator: increment as read, add, store.

2. Java Language Specification, 17.1 and 17.4.5: monitors, happens-before, data race definition, join ordering.
3. The Java Tutorials, Thread Interference (docs.oracle.com/javase/tutorial/essential/concurrency/interf.html).
4. Java SE API documentation: `AtomicInteger.incrementAndGet` and the `java.util.concurrent.atomic` package summary.
5. Goetz et al., Java Concurrency in Practice (2006), Chapter 2 (page numbers not verified).

12. Udit Contribution and Handoff

Implemented the three counters, the workload runner and 11 JUnit tests; executed 6 launches of 30 repetitions per implementation; documented methodology, environment and references in the repository. KT STATUS: PENDING.

Handoff to Adrika: synchronization is necessary because the lost update comes from unordered, non-atomic access to shared state, and it solves it by making the read-modify-write indivisible and ordering it across threads. A race condition is one such problem; locking itself can introduce others (such as deadlock), which are Adrika's scope and not analysed here.